

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Jorge Arun Zambrano Cedeño, Mg.

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS: DAMIAN JESUS CEDEÑO BRAVO

PARALELO: A

SEMESTRE: 5TO

ACTIVIDAD: 1



Actividades a desarrollar en la Unidad 1

Actividad # 1

(Desarrollo de la actividad propuesta)

Estudio de caso - Sistema de Gestión de Incidentes (Help Desk)

Diseñar la arquitectura lógica de un sistema de gestión de incidentes técnicos (Help Desk) para infraestructura de servidores que permita aplicar correctamente patrones de diseño de software y flujos de trabajo profesionales (Gitflow), fortaleciendo la capacidad de análisis, abstracción y buenas prácticas en el desarrollo de soluciones escalables, reutilizables y mantenibles.

Descripción de la Actividad

El estudiante deberá diseñar un sistema de Help Desk orientado a la gestión de fallos en un Data Center. El sistema debe permitir reportar incidentes (hardware, red, software), asignar técnicos especializados y gestionar estados de resolución. Para lograr una arquitectura clara y profesional, se deberá utilizar obligatoriamente un repositorio en GitHub bajo el modelo Gitflow y aplicar al menos tres patrones de diseño de software, justificando su elección y uso técnico.

1.- Análisis del problema

Identificar los actores principales

Los principales que van a actuar directamente con el sistema son los siguientes:

- Usuario Final: Personal que reporta los fallos en el sistema.
- Técnico de soporte: Se encarga de analizar y entender los que pasa a un componente para dar la solución más favorable.
- Administrador del sistema: Es responsable de gestionar todo el sistema, cada área del mismo. Identificar los actores principales
- Sistema de Notificaciones: Servicio que permite notificar o emitir alertas sobre nuevos tickets, fallos o cambios de estado.

Definir los requerimientos funcionales

A continuación, se detallan los requisitos funcionales del Sistema Help Desk:

- El sistema debe de permitir la autenticación de los datos como nombre de usuario y contraseña de cada usuario registrado.



- El sistema deberá de permitir reportar los fallos que ocurran dentro del sistema y a su vez indicando el tipo de fallo, ya sea de hardware, red o software
- El sistema debe de permitir que solo los administradores o los usuarios que estén autorizados puedan cambiar el estado de un fallo o incidente
- El sistema debe de permitir que se asigne un técnico de soporte especializado para cada incidente, dependiendo del tipo de incidente se asignará el técnico.
- El sistema debe de permitir a los administradores encargados, poder generar reportes de los diferentes tipos de incidentes, del técnico que realizo soporte y fecha de la labor realizada.

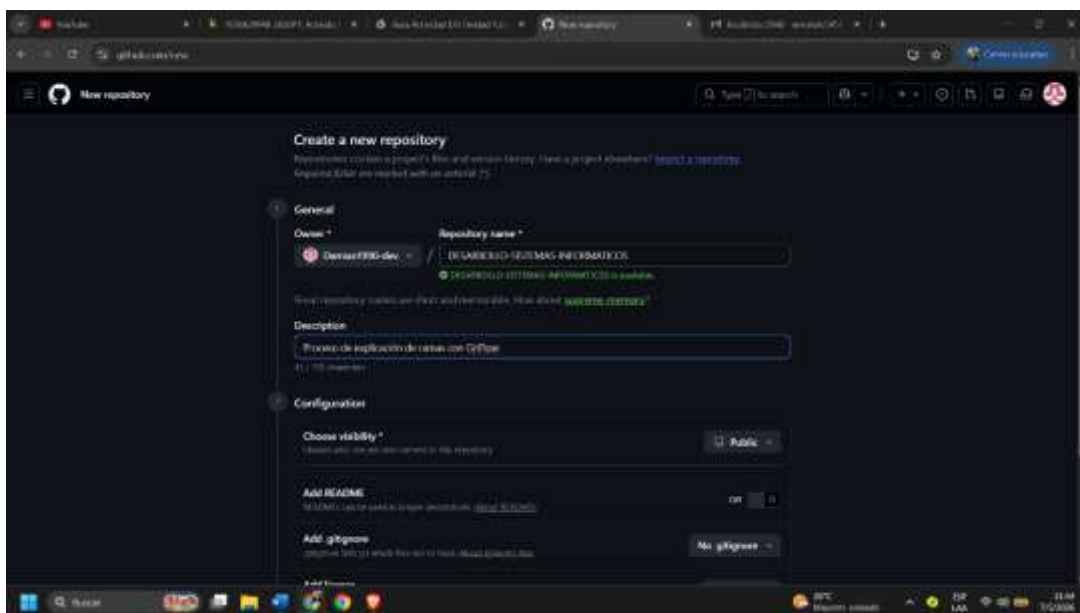
A continuación, se detallan los requisitos no funcionales del Sistema Help Desk:

- Escalabilidad: El sistema debe permitir agregar nuevos tipos de incidentes o roles sin modificar su estructura lógica.
- Mantenibilidad: El sistema debe de incluir una documentación básica donde se explique su funcionamiento.
- Trazabilidad de funciones: El proyecto debe de estar en un repositorio de github usando los modelos de flujo de GitFlow

2.- Diseño del sistema

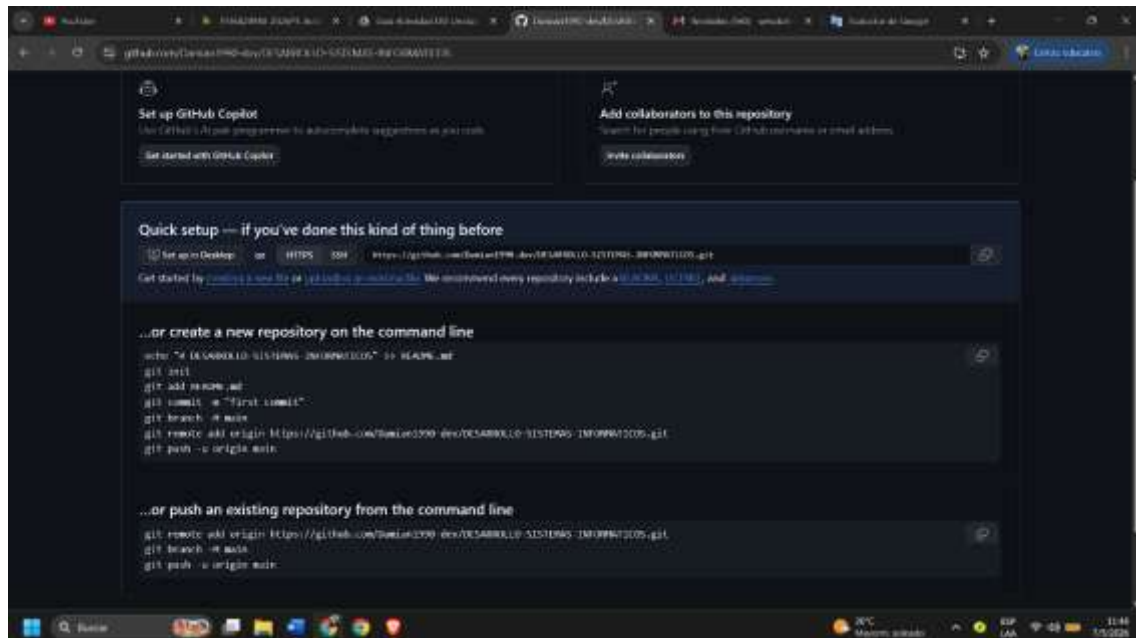
Crear un repositorio público en GitHub configurando la estructura de ramas Gitflow (master, develop y ramas feature/ para avances).

Primero vamos a crear un repositorio en GitHub, presionamos en botón Create repository o New, donde ubicamos un nombre y una descripción para el repositorio.

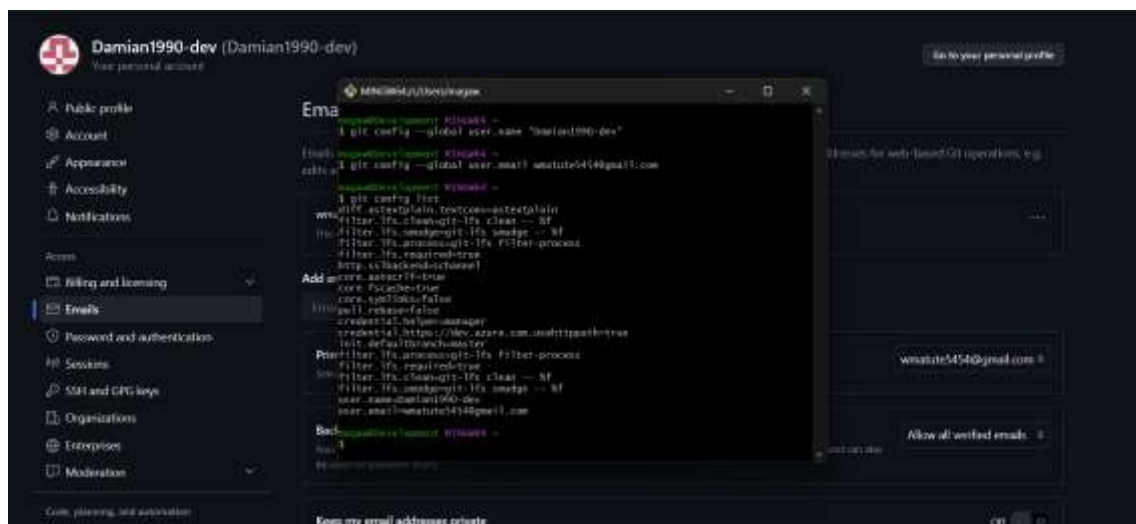




Una vez que esta creado el repositorio, se muestra una serie de comandos los cuales debemos de ingresar en la consola de git para subir cualquier tipo de documento o código.



Debemos de descargar Git y una vez realizada la instalación debemos de configurar el usuario y correo que va a estar vinculado con el git.





Una vez configurado ahora procedemos a subir cualquier archivo al repositorio que hemos creado, como prueba escogí un PDF para subirlo al repositorio.

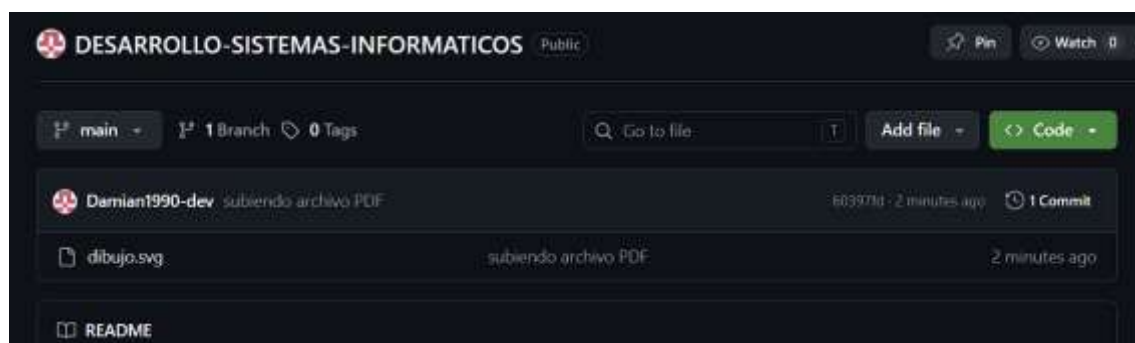
Nombre	Fecha de modificación	Tipo	Tamaño
git	7/5/2026 12:00	Carpeta de archivos	
dibujo	5/5/2026 17:03	Brave HTML Docu...	85 KB

```
mingw64 ~/OneDrive/Desktop/Purueba
$ git init
Initialized empty Git repository in C:/Users/magaw/OneDrive/Desktop/Purueba/.git/

mingw64 ~/OneDrive/Desktop/Purueba (master)
$ git add
warning: in the working copy of 'dibujo.svg', LF will be replaced by CRLF the next time git touches it.
$ git commit -m "subiendo archivo PDF"
[master (root-commit) 60397fd] subiendo archivo PDF
1 file changed, 41 insertions(+)
create mode 100644 dibujo.svg

mingw64 ~/OneDrive/Desktop/Purueba (master)
$ git branch -M main
$ git remote add origin https://github.com/Damian1990-dev/DESARROLLO-SISTEMAS-INFORMATICOS.git

mingw64 ~/OneDrive/Desktop/Purueba (main)
$ git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 31.79 KiB | 2.89 MiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Damian1990-dev/DESARROLLO-SISTEMAS-INFORMATICOS.git
 * [new branch]    main -> main
branch 'main' set up to track 'origin/main'.
```



Verificamos las ramas actuales que cuenta nuestro proyecto en GitHub, ingresamos en comando git Branch, actualmente contamos con una sola rama main

```
magaw@Development MINGW64 ~/OneDrive/Desktop/Purueba (main)
$ git branch
* main
```

Vamos a configurar la estructura de ramas Gitflow, rama master, develop y feature. En la consola de git ingresamos los siguientes comandos estando dentro del proyecto que se ha subido.



git checkout -b develop: Este comando crea la rama develop dentro del repositorio, una vez que se haya creado se debe de ingresar el comando git push -u origin develop para integrarla al GitHub.

```
magaw@Development MINGW64 ~/OneDrive/Desktop/Purueba (main)
$ git checkout -b develop
Switched to a new branch 'develop'

magaw@Development MINGW64 ~/OneDrive/Desktop/Purueba (develop)
$ git push -u origin develop
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'develop' on GitHub by visiting:
remote:   https://github.com/Damian1990-dev/DESARROLLO-SISTEMAS-INFORMATICOS/
remote: pull/new/develop
remote:
remote: To https://github.com/Damian1990-dev/DESARROLLO-SISTEMAS-INFORMATICOS.git
* [new branch]      develop -> develop
branch 'develop' set up to track 'origin/develop'.

magaw@Development MINGW64 ~/OneDrive/Desktop/Purueba (develop)
```

git checkout -b feature/update: Este comando crea la rama feature que se llama update, donde se van a crear nuevas funcionales dependiente de lo que se requiera, después se debe de subir al github haciendo un push

```
magaw@Development MINGW64 ~/OneDrive/Desktop/Purueba (develop)
$ git checkout -b feature/update
Switched to a new branch 'feature/update'

magaw@Development MINGW64 ~/OneDrive/Desktop/Purueba (feature/update)
$ git push -u origin feature/update
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/update' on GitHub by visiting:
remote:   https://github.com/Damian1990-dev/DESARROLLO-SISTEMAS-INFORMATICOS/
remote: pull/new/feature/update
remote:
remote: To https://github.com/Damian1990-dev/DESARROLLO-SISTEMAS-INFORMATICOS.git
* [new branch]      feature/update -> feature/update
branch 'feature/update' set up to track 'origin/feature/update'.

magaw@Development MINGW64 ~/OneDrive/Desktop/Purueba (feature/update)
$ |
```

Una vez realizado este proceso volvemos a la rama de desarrollo que es develop e ingresamos el comando git branch para verificar las ramas actuales en el proyecto.

Rama master: main

Rama de desarrollo: develop

Rama de nuevas funcionalidades: feature

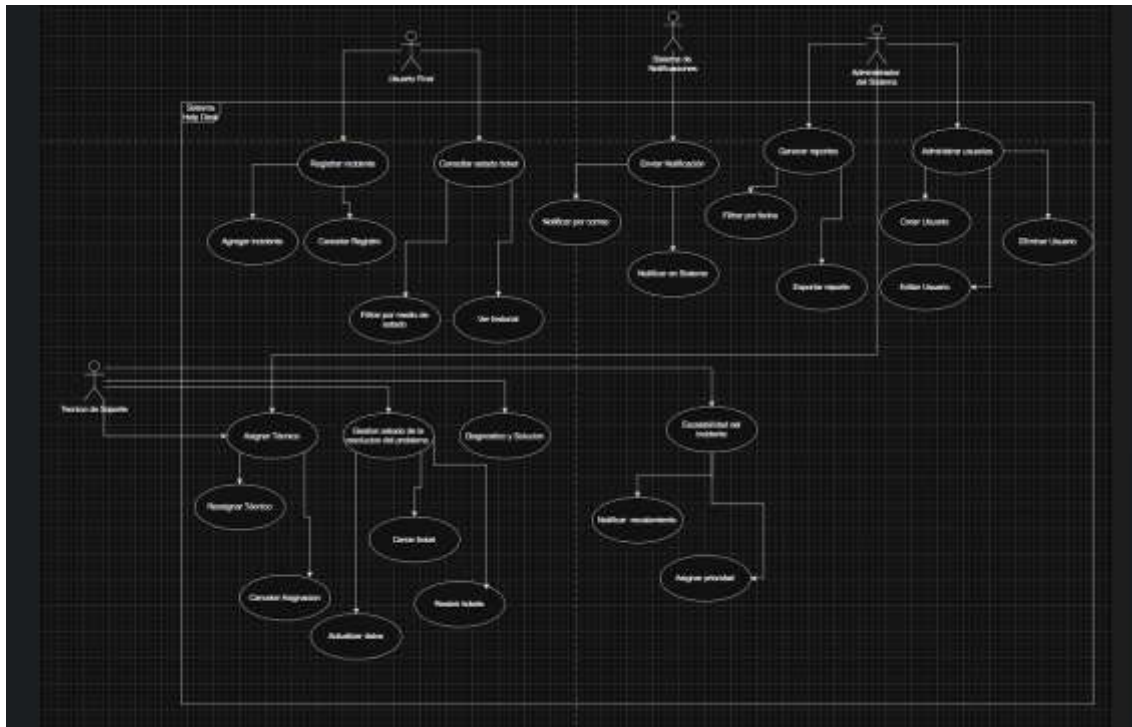
```
magaw@Development MINGW64 ~/OneDrive/Desktop/Purueba (feature/update)
$ git checkout develop
Switched to branch 'develop'
Your branch is up to date with 'origin/develop'.

magaw@Development MINGW64 ~/OneDrive/Desktop/Purueba (develop)
$ git branch
* develop
  feature/update
  main

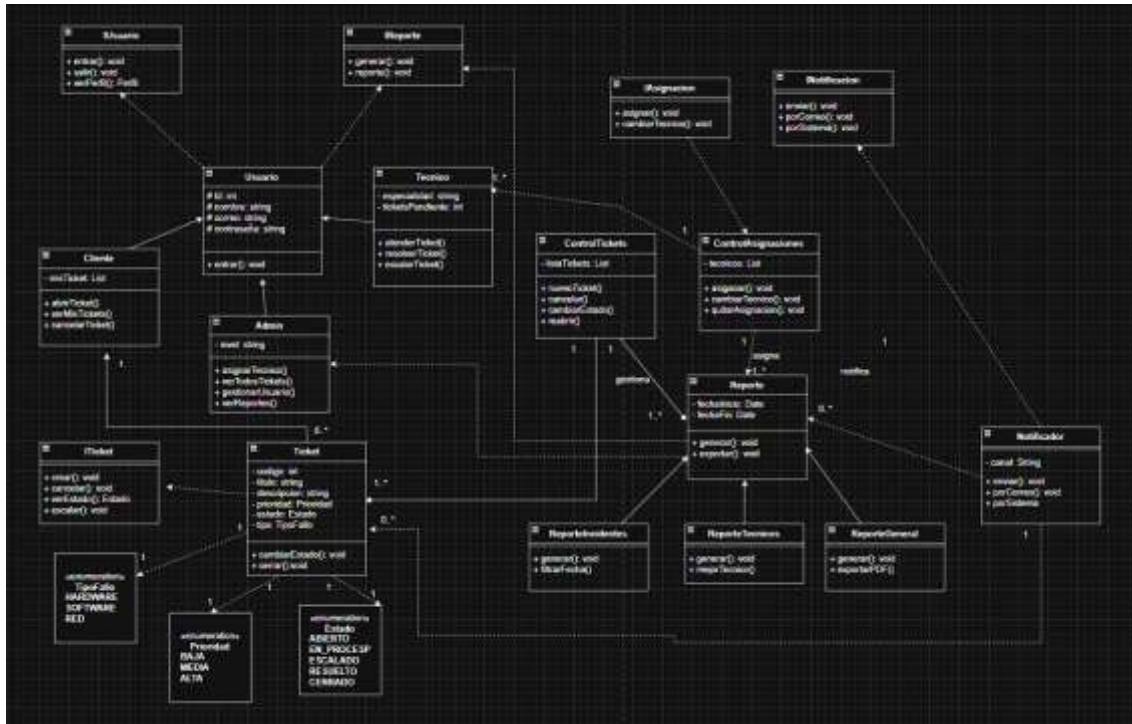
magaw@Development MINGW64 ~/OneDrive/Desktop/Purueba (develop)
```

Crear un diagrama de casos de uso para entender las interacciones del personal de TI con la infraestructura.

A continuación, se detalla el diagrama de caso de uso UML con su respectivo desarrollo de los actores principales con sus funcionalidades:



Realizar un diagrama de clases UML donde se evidencien los objetos y su relación bajo principios SOLID.



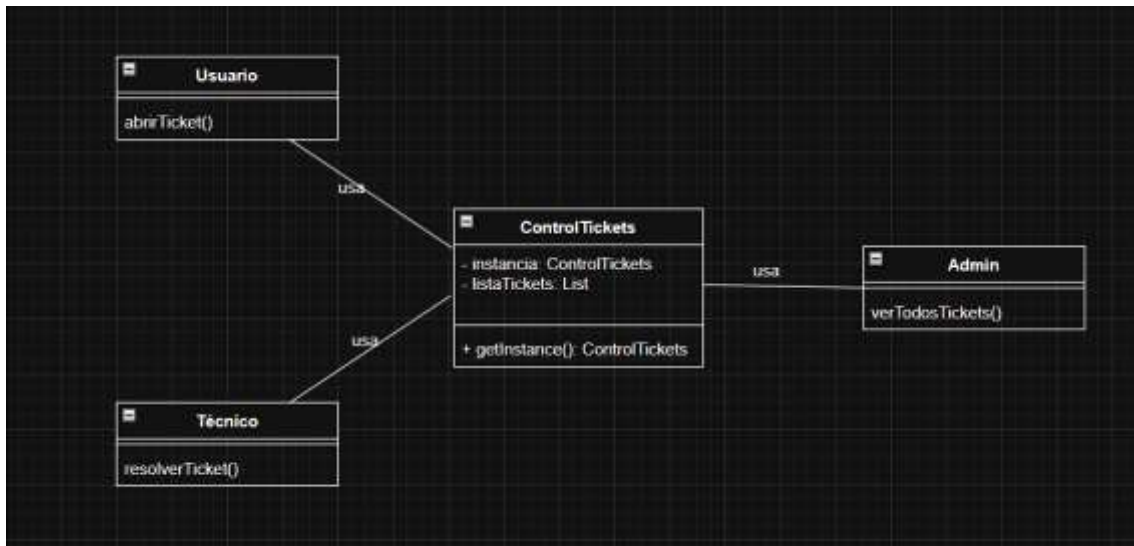
Diseñar la arquitectura general del sistema integrando el flujo de versiones

La arquitectura general del Sistema Help Desk está dividida en tres partes principales que trabajan juntas para que el sistema funcione correctamente. La primera parte es la interfaz de usuario, que es donde los actores del sistema, como el Usuario Final, el Técnico de Soporte y el Administrador, pueden interactuar con el sistema según lo que cada uno necesite hacer, ya sea reportar un incidente, atenderlo o revisar reportes. La segunda parte es la lógica del sistema, que se encarga de procesar todo lo que ocurre dentro del sistema, como asignar automáticamente un técnico según el tipo de fallo que se reportó, ya sea de hardware, red o software, cambiar el estado de los tickets y enviar notificaciones cuando hay algún cambio importante. La tercera parte es la base de datos, donde se guarda toda la información de los incidentes, los usuarios, los técnicos y el historial de cada caso atendido. Estas tres partes están conectadas entre sí y además se integran con el flujo de trabajo Gitflow que se configuró en GitHub, ya que cada vez que se quiere agregar una nueva funcionalidad al sistema se crea una rama feature, se prueba en la rama develop y cuando todo está bien se pasa a la rama master, que es la versión final del proyecto. De esta manera, el sistema puede crecer y mejorar con el tiempo sin afectar lo que ya está funcionando, lo cual está directamente relacionado con los requerimientos de escalabilidad y mantenibilidad que se definieron al inicio de la actividad.

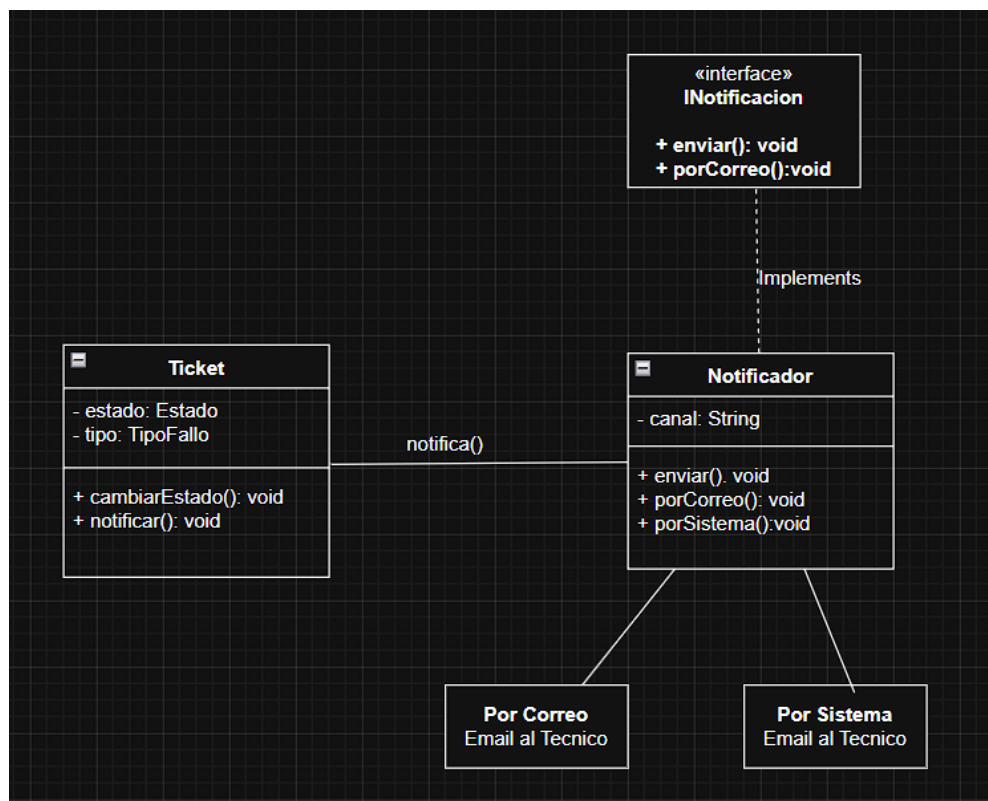
3. Aplicación de patrones de diseño

Utilizar al menos tres patrones. Algunos recomendados:

Singleton: Para la instancia central del motor de gestión de tickets y conexión a datos



Observer: Indispensable para el sistema de notificaciones automáticas a técnicos ante nuevos incidentes (Base de la programación reactiva).





4.- Documentación del diseño

Explicar el propósito de cada patrón utilizado y cómo resuelve un problema específico del Help Desk

Patrón Singleton

El patrón Singleton se utilizó para resolver un problema muy concreto dentro del sistema Help Desk: evitar que existan varias copias de la lista de tickets funcionando al mismo tiempo. Sin este patrón, podría darse el caso de que el Usuario Final abra un ticket desde su sesión, el Técnico lo busque desde la suya y el Administrador lo revise desde otra, y cada uno esté viendo una versión diferente de la información porque el sistema creó múltiples instancias de la clase ControlTickets sin ningún control. Esto causaría inconsistencias graves como tickets duplicados, estados incorrectos o asignaciones que no coinciden.

Patrón Observador

El patrón Observer se utilizó para resolver el problema de las notificaciones automáticas dentro del sistema Help Desk. Antes de aplicar este patrón, la única forma de saber si un ticket había cambiado de estado era que el técnico o el administrador revisaran manualmente el sistema cada cierto tiempo, lo cual es poco eficiente y propenso a errores. Con el Observer, la clase Ticket actúa como el elemento que se observa, y cada vez que su estado cambia, por ejemplo de ABIERTO a EN_PROCESO o de EN_PROCESO a RESUELTO, automáticamente llama al método notificar()

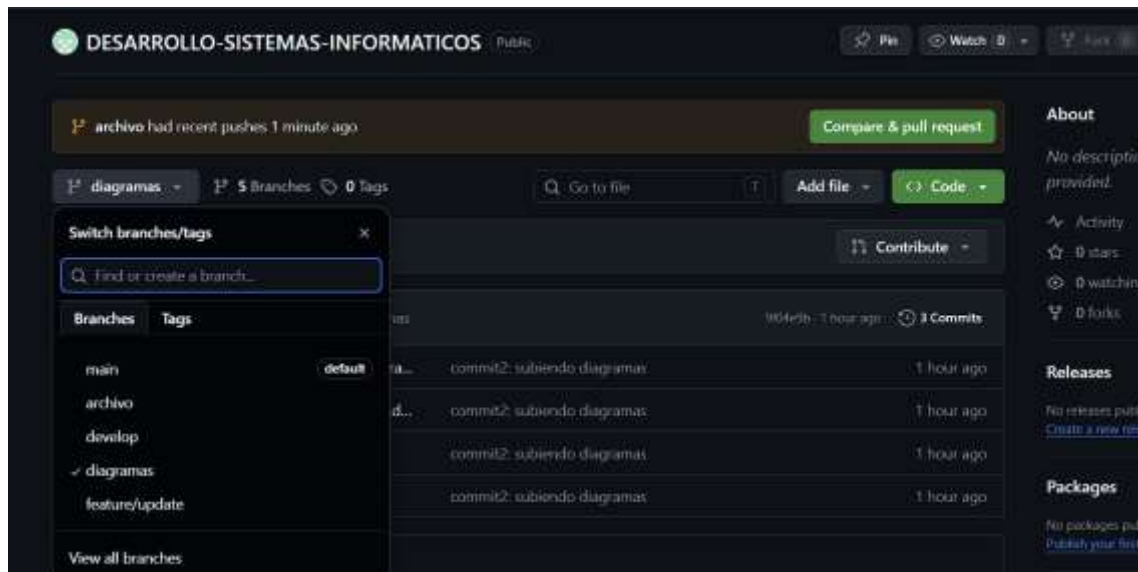
Justificar el uso de Gitflow: Incluir el enlace al repositorio de GitHub y capturas de pantalla del historial de ramas/commits que evidencien el flujo de trabajo

Enlace de repositorio

<https://github.com/DevStack212005/DESARROLLO-SISTEMAS-INFORMATICOS.git>



Ramas Creada



```
magaw@Development MINGW64 ~/OneDrive/Desktop/ACTIVIDAD 1 - DSI - DAMIAN (archivo)
$ git branch
* archivo
  develop
  diagramas
  feature/update
  main

magaw@Development MINGW64 ~/OneDrive/Desktop/ACTIVIDAD 1 - DSI - DAMIAN (archivo)
```

Commit realizados

